Task 1 – Text Composables Understanding & Exploration

Priyanka K

1. Definition – What is a `@Composable` Function?

A `@Composable` function is a special Kotlin function used in Jetpack Compose to define a piece of user interface (UI). It describes what should appear on the screen and how it should be arranged. Instead of using XML layouts, Compose builds the UI directly through composable functions written in Kotlin.

2. Key Text Composables Used in the Implementation

1. Text

What it does in UI layout: Displays text content on the screen.

Example use-case in the app: Displaying the message "Happy Birthday Sam!" and the sender name "From Emma".

2. Column

What it does in UI layout: Arranges UI elements vertically from top to bottom.

Example use-case in the app: Stacking the birthday message and sender name one below the other inside the `GreetingText` composable.

3. Box

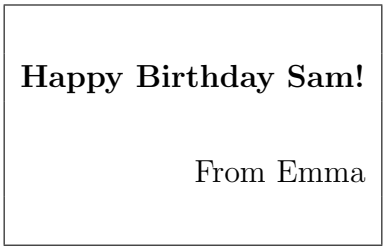
What it does in UI layout: Stacks UI elements on top of each other, allowing overlay design.

Example use-case in the app: Placing the birthday message text on top of the background image inside the `GreetingImage` composable.

3. How Column Affects Text Appearance

In the `GreetingText` composable, the `Column` layout is used with `verticalArrangement = Arrangement.Center`. This ensures that the text elements are arranged vertically and centered on the screen. The sender text is aligned to the end using `Alignment.End`, which positions it toward the right side of the layout. Without using `Column`, the text elements would not be properly structured or aligned.

4. Wireframe of the Screen



Composables Used:

- `Box` – Main layout container
- `Image` – Background image
- `Column` – Vertical arrangement of text
- `Text` – Displaying message and sender name

5. Implementation Output

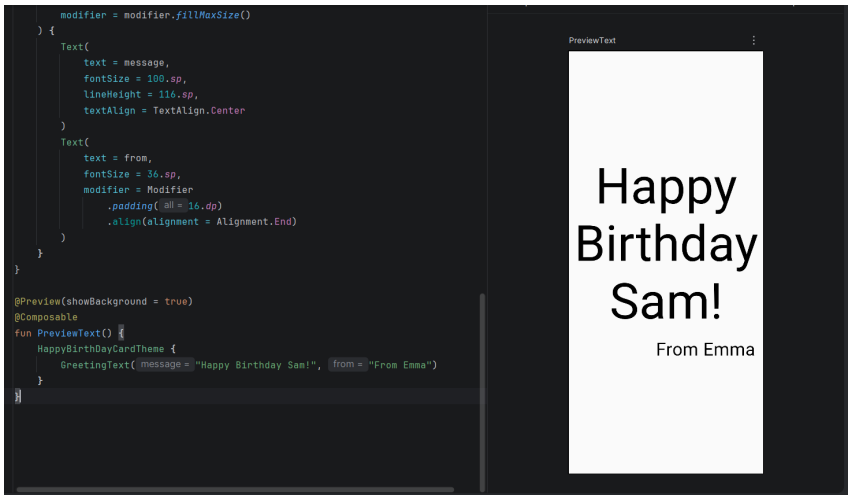


Figure 1: GreetingText Composable Output (Text Only Layout)

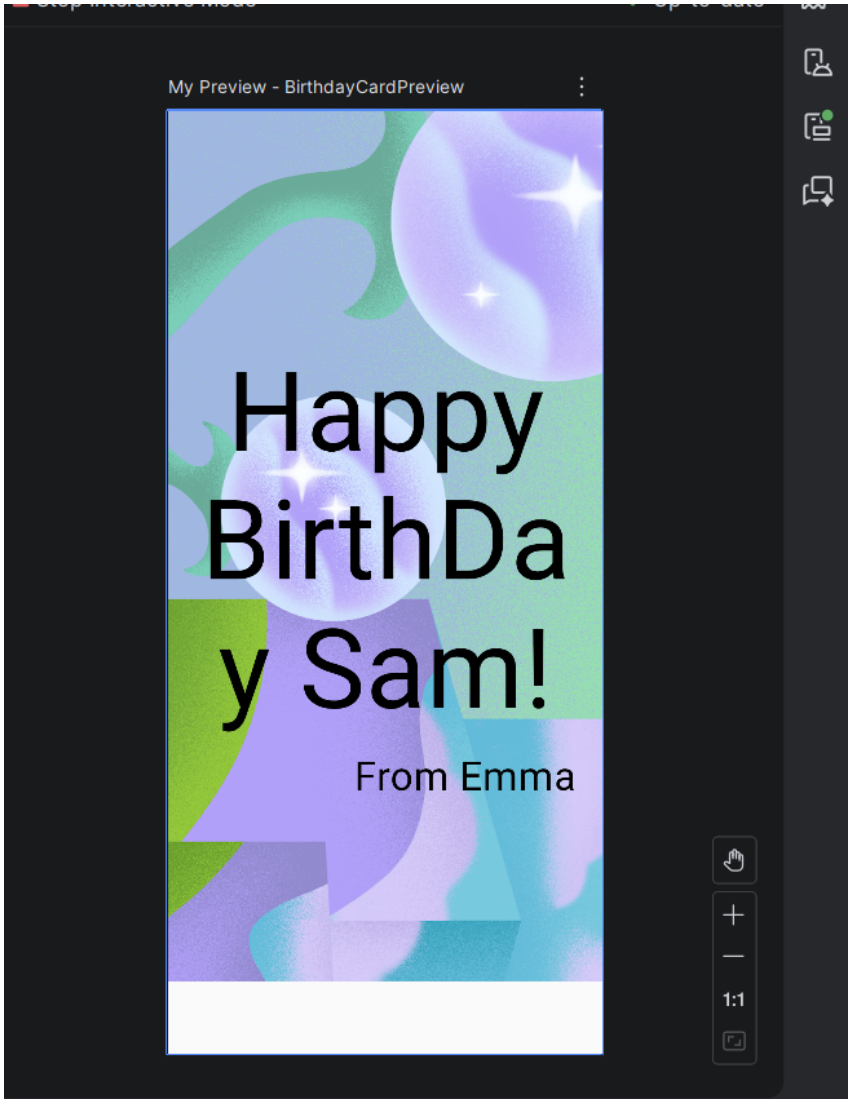


Figure 2: GreetingImage Composable Output (Image with Text Overlay)

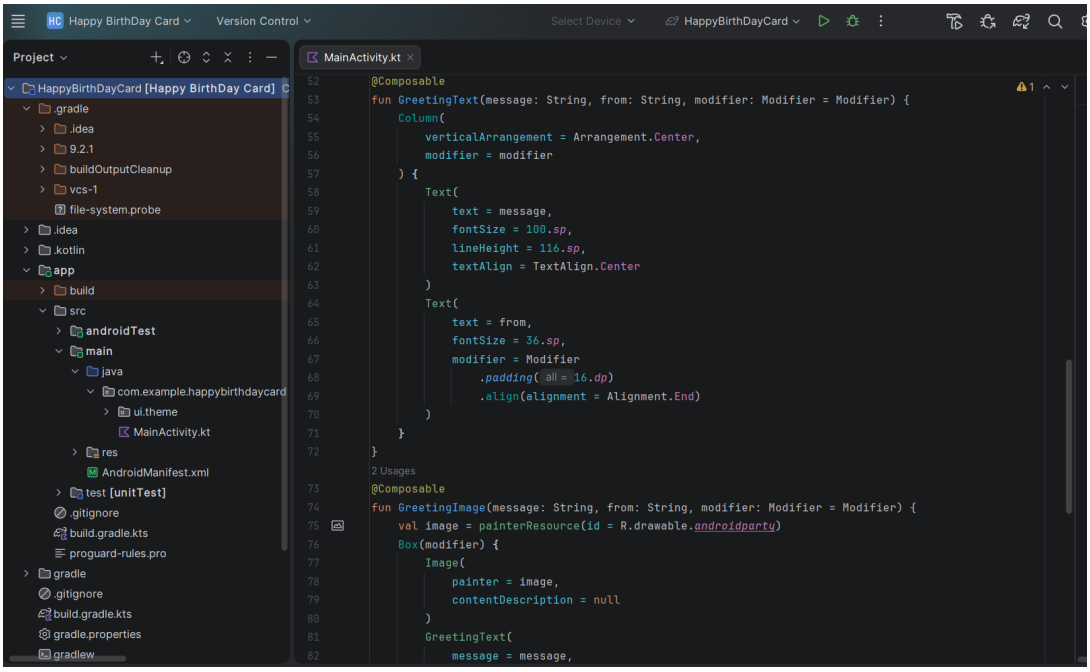


Figure 3: Composable Structure Showing GreetingText and GreetingImage

6. Reflection

Planning the text layout before coding helps in understanding how UI elements will be structured and arranged. It improves clarity in alignment and spacing decisions. Visualizing the layout beforehand reduces implementation errors and unnecessary modifications later. It also ensures better organization of composables and cleaner UI design in Jetpack Compose.